

SHKeeper + Tron Daemon — Інструкція для DevOps

Дата: 2026-03-24 **Автор:** Exchange Backend Team **Статус:** Готово до розгортання

Зміст

- [Загальний огляд](#)
- [Репозиторії та вихідний код](#)
- [Архітектура контейнерів](#)
- [Системні вимоги](#)
- [Покрокове розгортання](#)
- [Змінні оточення \(Environment Variables\)](#)
- [Порти та мережева конфігурація](#)
- [Зв'язок з Exchange Backend](#)
- [Nginx reverse proxy](#)
- [SSL сертифікати](#)
- [Перший запуск та ініціалізація](#)
- [Перемикання мереж \(Testnet ↔ Mainnet\)](#)
- [Моніторинг та логування](#)
- [Бекапи та відновлення](#)
- [Оновлення](#)
- [Troubleshooting](#)
- [Чеклісти](#)

1. Загальний огляд

SHKeeper — це open-source self-hosted payment processor для криптовалют від [vsys-host](#). Він приймає криптоплатежі, генерує гаманці, сканує блокчейн на вхідні транзакції, та виконує виплати (payouts).

Наша конфігурація: використовуємо SHKeeper **виключно для Tron мережі** (TRX + USDT TRC-20). Інші блокчейни (BTC, ETH, BNB, SOL, Polygon, Avalanche) **вимкнені**.

Компоненти, які розгортаються:

Компонент	Опис	Образ
shkeeper	Головний Flask додаток (UI + REST API)	Білдиться з Dockerfile (Python 3.13)
tron-shkeeper	Tron daemon — сканер блоків, генерація гаманців	Білдиться з daemons/tron-shkeeper/Dockerfile (Python 3.13)
tron-shkeeper-celery	Celery worker для Tron — фонові задачі (payouts, sweeps)	Той самий образ що й tron-shkeeper
shkeeper-redis	Redis 7 — Celery broker	redis:7-alpine

Всього: 4 контейнери.

2. Репозиторії та вихідний код

Основні репозиторії (GitHub, vsys-host):

Репозиторій	URL	Що це
shkeeper.io	https://github.com/vsys-host/shkeeper.io.git	Головний додаток (Flask UI + API)
tron-shkeeper	https://github.com/vsys-host/tron-shkeeper.git	Tron blockchain daemon

Структура файлів після клонування:

shkeeper/	# ← корінь SHKeeper
├─ docker-compose.yml	# Docker Compose для всіх контейнерів
├─ Dockerfile	# Dockerfile для головного shkeeper app
├─ .env.example	# Приклад змінних оточення
├─ requirements.txt	# Python залежності головного app
├─ shkeeper/	# Flask додаток (вихідний код)
│ └─ __init__.py	
│ └─ models.py	
│ └─ wallet_utils.py	
│ └─ ...	
├─ migrations/	# Alembic міграції (SQLite)
├─ daemons/	# Підкаталог з daemon-сервісами
│ └─ tron-shkeeper/	# ← Клонований tron-shkeeper репозиторій
│ └─ Dockerfile	
│ └─ requirements.txt	
│ └─ run.py	# Entry point для gunicorn
│ └─ celery_worker.py	# Entry point для Celery
│ └─ app/	# Код даємон'а
│ └─ config.py	# Конфігурація (pydantic-settings)
│ └─ block_scanner.py	# Сканер блоків
│ └─ wallet.py	# Управління гаманцями
│ └─ tasks.py	# Celery tasks
│ └─ api/	# REST API endpoints
│ └─ views.py	
│ └─ payout.py	
│ └─ metrics.py	
└─ docs/	

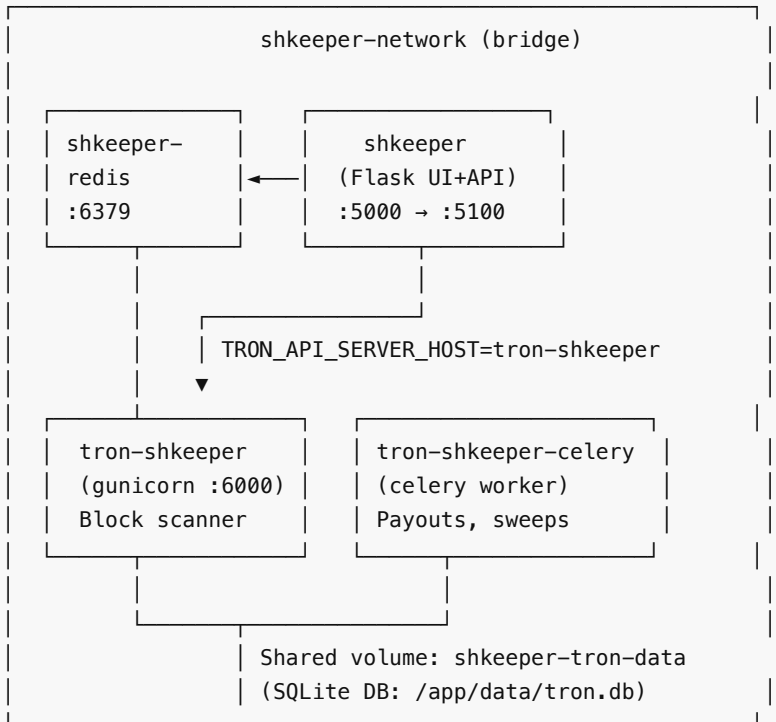
Як отримати код:

```
# 1. Клонувати головний shkeeper
git clone https://github.com/vsys-host/shkeeper.io.git shkeeper
cd shkeeper

# 2. Клонувати tron даємон в підкаталог daemons/
mkdir -p daemons
git clone https://github.com/vsys-host/tron-shkeeper.git daemons/tron-shkeeper
```

⚠ **Важливо:** Ми не форкали ці репозиторії. Код клонується напряму з upstream. Всі наші зміни (`docker-compose.yml`, `env`-конфіг) живуть у нашому монорепо `trocador/shkeeper/`.

3. Архітектура контейнерів



Зовнішні з'єднання:

- ← `buysell-service` (Exchange Backend) звертається до `shkeeper:5100`
- `tron-shkeeper` звертається до `FULLNODE_URL` (Tron RPC node)

Потік даних:

- Депозити:** `tron-shkeeper` сканує блоки Tron → знаходить транзакцію на наш гаманець → повідомляє `shkeeper` → `shkeeper` шле `webhook` на `buysell-service`
- Виплати:** `buysell-service` → POST до `shkeeper` API → `shkeeper` передає задачу в `tron-shkeeper-celery` → `celery` виконує Tron-транзакцію → `webhook` назад

4. Системні вимоги

Ресурс	Мінімум	Рекомендовано
RAM	512 MB	1 GB
CPU	1 core	2 cores
Диск	2 GB (образи + дані)	5 GB (з запасом)
Docker	24.0+	Остання стабільна

Docker Compose	v2.20+	Остання
----------------	--------	---------

SHKeeper з одним даємон'ом (Tron) використовує ~~300–500 MB RAM~~. Це значно менше ніж повна конфігурація з 7 даємонами (2-3 GB).

5. Покрокове розгортання

Крок 1: Підготовка директорії

```
# На сервері, обираємо місце для SHKeeper
# Рекомендовано: поруч з Exchange Backend
mkdir -p /opt/shkeeper
cd /opt/shkeeper
```

Крок 2: Клонування репозиторіїв

```
# Головний SHKeeper
git clone https://github.com/vsys-host/shkeeper.io.git .

# Tron daemon
mkdir -p daemons
git clone https://github.com/vsys-host/tron-shkeeper.git daemons/tron-shkeeper
```

Крок 3: Створення docker-compose.yml

Створити файл `docker-compose.yml` з наступним вмістом:

```
services:

# =====
# Redis – Celery broker
# =====

shkeeper-redis:
  image: redis:7-alpine
  container_name: shkeeper-redis
  ports:
    - '6380:6379'          # 6380 зовні, щоб не конфліктувати
  volumes:
    - shkeeper-redis-data:/data
  restart: unless-stopped
  healthcheck:
    test: ['CMD', 'redis-cli', 'ping']
    interval: 5s
    timeout: 3s
    retries: 5
  networks:
    - shkeeper-network
```

```

# =====
# SHKeeper Main App (Flask UI + REST API)
# =====

shkeeper:
  build:
    context: .
    dockerfile: Dockerfile
  container_name: shkeeper
  ports:
    - '5100:5000'          # UI + API порт
  environment:
    # --- Режим роботи ---
    - DEV_MODE=0           # ⚠ PRODUCTION: поставити 0
    - FORCE_WALLET_ENCRYPTION=0
    # --- Вимикаємо все крім Tron ---
    - LTC_WALLET=disabled
    - DOGE_WALLET=disabled
    - ETH_WALLET=disabled
    - ETH_USDT_WALLET=disabled
    - ETH_USDC_WALLET=disabled
    - BNB_WALLET=disabled
    - BNB_USDT_WALLET=disabled
    - BNB_USDC_WALLET=disabled
    - SOL_WALLET=disabled
    - SOLANA_USDT_WALLET=disabled
    - SOLANA_USDC_WALLET=disabled
    - MATIC_WALLET=disabled
    - POLYGON_USDT_WALLET=disabled
    - POLYGON_USDC_WALLET=disabled
    - AVAX_WALLET=disabled
    - AVALANCHE_USDT_WALLET=disabled
    - AVALANCHE_USDC_WALLET=disabled
    # --- Tron: YBIMKHEHO ---
    - TRX_WALLET=enabled
    - USDT_WALLET=enabled
    - USDC_WALLET=disabled  # USDC не використовуємо
    # --- Tron daemon hostname ---
    - TRON_API_SERVER_HOST=tron-shkeeper
  volumes:
    - shkeeper-data:/shkeeper.io/instance
  restart: unless-stopped
  depends_on:
    shkeeper-redis:
      condition: service_healthy
  networks:
    - shkeeper-network

# =====
# Tron Daemon (TRX, USDT TRC-20)
# =====

```

```

tron-shkeeper:
  build:
    context: ./daemons/tron-shkeeper
  container_name: tron-shkeeper
  command: gunicorn -b 0.0.0.0:6000 --timeout 120 --workers 1 run:server
  environment:
    - TRON_NETWORK=main # "main" для mainnet, "nile"
для testnet
    - FULLNODE_URL=http://fullnode.tron.shkeeper.io # Або ваша приватна нода
    - DB_URI=sqlite:///data/tron.db
    - REDIS_HOST=shkeeper-redis
    - SHKEEPER_HOST=shkeeper:5000
    - SHKEEPER_BACKEND_KEY=shkeeper
    - BTC_USERNAME=shkeeper
    - BTC_PASSWORD=shkeeper
    - BLOCK_SCANNER_MAX_BLOCK_CHUNK_SIZE=5
  volumes:
    - shkeeper-tron-data:/app/data
  restart: unless-stopped
  depends_on:
    shkeeper-redis:
      condition: service_healthy
  networks:
    - shkeeper-network

# =====
# Tron Celery Worker (payouts, sweeps)
# =====

tron-shkeeper-celery:
  build:
    context: ./daemons/tron-shkeeper
  container_name: tron-shkeeper-celery
  command: celery -A celery_worker.celery worker --loglevel=info --without-
heartbeat
  environment:
    - C_FORCE_ROOT=1 # Дозволяє Celery працювати
під root
    - TRON_NETWORK=main
    - FULLNODE_URL=http://fullnode.tron.shkeeper.io
    - DB_URI=sqlite:///data/tron.db
    - REDIS_HOST=shkeeper-redis
    - SHKEEPER_HOST=shkeeper:5000
    - SHKEEPER_BACKEND_KEY=shkeeper
    - BTC_USERNAME=shkeeper
    - BTC_PASSWORD=shkeeper
    - BLOCK_SCANNER_MAX_BLOCK_CHUNK_SIZE=5
  volumes:
    - shkeeper-tron-data:/app/data # Ділить volume з tron-
shkeeper
  restart: unless-stopped

```

```

    depends_on:
      shkeeper-redis:
        condition: service_healthy
    networks:
      - shkeeper-network

# =====
# Volumes & Networks
# =====

volumes:
  shkeeper-data:      # Дані головного shkeeper (SQLite, wallets, config)
  shkeeper-redis-data: # Redis persistence
  shkeeper-tron-data:  # Tron daemon data (SQLite DB, wallet keys)

networks:
  shkeeper-network:
    driver: bridge

```

Крок 4: Білд та запуск

```

cd /opt/shkeeper

# Білд всіх образів
docker compose build

# Запуск у фоні
docker compose up -d

# Перевірити статус
docker compose ps

```

Крок 5: Перевірка здоров'я контейнерів

```

# Всі 4 контейнери мають бути Up
docker compose ps

# Очікуваний вивід:
# NAME                STATUS          PORTS
# shkeeper             Up             0.0.0.0:5100->5000/tcp
# shkeeper-redis       Up (healthy)    0.0.0.0:6380->6379/tcp
# tron-shkeeper        Up
# tron-shkeeper-celery Up

# Перевірити логи
docker logs shkeeper --tail=50
docker logs tron-shkeeper --tail=50
docker logs tron-shkeeper-celery --tail=50

```

6. Змінні оточення (Environment Variables)

Головний SHKeeper (shkeeper контейнер)

Змінна	Значення (prod)	Значення (dev)	Опис
DEV_MODE	0	1	Dev-режим (спрощений логін, менше валідацій)
DEV_MODE_ENC_PW	—	devpassword123	Автопароль для wallet encryption у dev-режимі
FORCE_WALLET_ENCRYPTION	0	0	Примусове шифрування гаманців
TRX_WALLET	enabled	enabled	Увімкнути підтримку TRX
USDT_WALLET	enabled	enabled	Увімкнути підтримку USDT TRC-20
USDC_WALLET	disabled	disabled	USDC TRC-20 (не використовуємо)
TRON_API_SERVER_HOST	tron-shkeeper	tron-shkeeper	Hostname Tron daemon'a в Docker мережі
Всі інші *_WALLET	disabled	disabled	Відключені блокчейни

Tron Daemon (tron-shkeeper + tron-shkeeper-celery)

Змінна	Значення (prod/mainnet)	Значення (dev/testnet)	
TRON_NETWORK	main	nile	М Т а
FULLNODE_URL	http://fullnode.tron.shkeeper.io	https://nile.trongrid.io	Ф Т
DB_URI	sqlite:///data/tron.db	sqlite:///data/tron.db	Л С
REDIS_HOST	shkeeper-redis	shkeeper-redis	Т Ф Г М
SHKEEPER_HOST	shkeeper:5000	shkeeper:5000	Т Г С
SHKEEPER_BACKEND_KEY	shkeeper	shkeeper	А З С С

BTC_USERNAME	shkeeper	shkeeper	Л с (п
BTC_PASSWORD	shkeeper	shkeeper	Г А
C_FORCE_ROOT	1	1	Т с к Е С с
BLOCK_SCANNER_MAX_BLOCK_CHUNK_SIZE	5	5	к Б п с

Альтернативні Tron RPC endpoints (якщо потрібна своя нода):

Мережа	URL	Примітка
Mainnet (vsys-host публічна)	http://fullnode.tron.shkeeper.io	За замовчуванням
Mainnet (TronGrid, безкоштовно до ліміту)	https://api.trongrid.io	Rate-limited
Testnet Nile	https://nile.trongrid.io	Для тестування

*Для **production** рекомендується власна Tron full-node або платний план TronGrid/QuickNode для стабільності.*

7. Порти та мережева конфігурація

Зайняті порти

Сервіс	Порт на хості	Порт в контейнері	Протокол	Призначення
shkeeper (UI + API)	5100	5000	HTTP	Веб-інтерфейс + REST API для buysell-service
shkeeper-redis	6380	6379	TCP	Celery broker (Redis)
tron-shkeeper	— (тільки internal)	6000	HTTP	Tron daemon API (не виставляється назовні)
tron-shkeeper-celery	—	—	—	Worker, не має порту

Перевірка зайнятості портів перед запуском:

```
# Перевірити чи порти 5100 та 6380 вільні
ss -tlnp | grep -E ':(5100|6380)\s'

# Якщо зайняті — або змінити порти в docker-compose.yml,
# або зупинити конфліктуючий сервіс
```

Docker мережі

SHKeeper використовує ізольовану мережу `shkeeper-network` (bridge driver). Контейнери спілкуються між собою по hostname:

```
shkeeper-redis    → redis://shkeeper-redis:6379
tron-shkeeper     → http://tron-shkeeper:6000
shkeeper          → http://shkeeper:5000
```

8. Зв'язок з Exchange Backend

Exchange Backend (buysell-service) потребує доступу до SHKeeper API. Є два підходи:

Варіант А: Через хостовий порт (простіший)

buysell-service звертається до SHKeeper через порт 5100 на хості:

```
SHKEEPER_BASE_URL=http://host.docker.internal:5100 # якщо buysell в Docker
SHKEEPER_BASE_URL=http://localhost:5100           # якщо buysell на хості
SHKEEPER_BASE_URL=http://<server-ip>:5100         # якщо різні сервери
```

Варіант Б: Спільна Docker мережа (рекомендовано для production)

Додати зовнішню мережу в обох docker-compose файлах:

У SHKeeper docker-compose.yml:

```
networks:
  shkeeper-network:
    driver: bridge
  exchanger-network:
    external: true          # Підключаємось до мережі Exchange Backend

services:
  shkeeper:
    networks:
      - shkeeper-network
      - exchanger-network  # SHKeeper доступний в мережі бекенду
```

У Exchange Backend docker-compose.yml:

```
services:
  buysell-service:
    environment:
      - SHKEEPER_BASE_URL=http://shkeeper:5000 # Напрямку по hostname
```

Тоді buysell-service зможе звертатись до `http://shkeeper:5000` напряму.

Змінні для Exchange Backend (.env):

Змінна	Опис	Приклад
SHKEEPER_BASE_URL	URL SHKeeper API	http://shkeeper:5000 або http://localhost:5100
SHKEEPER_API_KEY	API ключ (генерується в UI SHKeeper)	sk_live_abc123...
SHKEEPER_PAYOUT_LOGIN	Логін для payouts (HTTP Basic Auth)	admin
SHKEEPER_PAYOUT_PASSWORD	Пароль для payouts	<secure-password>
SHKEEPER_WEBHOOK_CALLBACK_URL	URL куди SHKeeper шле deposit webhooks	http://buysell- service:4337/webhooks/shkeeper/deposit
SHKEEPER_PAYOUT_CALLBACK_URL	URL куди SHKeeper шле payout webhooks	http://buysell- service:4337/webhooks/shkeeper/payout

⚠ SHKEEPER_WEBHOOK_CALLBACK_URL — це URL, який SHKeeper використовує для виклику buysell-service. Він повинен бути доступний з контейнера shkeeper. Якщо сервіси в спільній Docker мережі — використовуйте hostname. Якщо ні — використовуйте IP сервера.

9. Nginx reverse proxy

Якщо SHKeeper UI потрібно виставити через nginx (наприклад, для адмін-доступу ззовні):

Конфігурація nginx:

```
# /etc/nginx/conf.d/shkeeper.techchaininnovations.com.conf

upstream shkeeper_upstream {
  server 127.0.0.1:5100;
}

server {
  listen 80;
  server_name shkeeper.techchaininnovations.com;
  return 301 https://$host$request_uri;
```

```

}

server {
    listen 443 ssl http2;
    server_name shkeeper.techchaininnovations.com;

    ssl_certificate
/etc/letsencrypt/live/shkeeper.techchaininnovations.com/fullchain.pem;
    ssl_certificate_key
/etc/letsencrypt/live/shkeeper.techchaininnovations.com/privkey.pem;

    # Security: обмежити доступ до UI по IP (рекомендовано!)
    # allow 1.2.3.4;      # IP адміністратора
    # deny all;

    location / {
        proxy_pass http://shkeeper_upstream;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

```

Створення:

```

# 1. Створити конфіг nginx
sudo nano /etc/nginx/conf.d/shkeeper.techchaininnovations.com.conf

# 2. Перевірити синтаксис
sudo nginx -t

# 3. Перезавантажити nginx
sudo systemctl reload nginx

```

10. SSL сертифікати

```

# Генерація SSL через certbot (якщо SHKeeper виставляється через nginx)
sudo certbot certonly --nginx -d shkeeper.techchaininnovations.com

# Автоматичне оновлення сертифікатів (certbot зазвичай додає cron автоматично)
sudo certbot renew --dry-run

```

Примітка: SSL потрібен тільки якщо SHKeeper UI виставляється назовні. Для внутрішнього зв'язку `buysell-service` → `shkeeper` SSL не потрібен.

11. Перший запуск та ініціалізація

Після `docker compose up -d` потрібно виконати початкову настройку через UI:

Крок 1: Відкрити UI

```
http://<server-ip>:5100
```

При першому запуску SHKeeper перенаправить на `/set-password`.

Крок 2: Встановити пароль адміністратора

- Ввести та підтвердити пароль
- Цей пароль використовується для логіну в UI та як `SHKEEPER_PAYOUT_PASSWORD`

Крок 3: Wallet Encryption

Після логіну з'явиться запит на wallet encryption:

- **Для production:** Увімкнути та встановити надійний пароль шифрування
- **Для dev/testing:** Можна пропустити (skip)

Поки wallet encryption не налаштована, Tron daemon буде в стані "pending" і не зможе генерувати гаманці.

Крок 4: Отримати API Key

1. Перейти в **Wallets** → **USDT** → **Manage**
2. Скопіювати **API Key** — це буде значення для `SHKEEPER_API_KEY` в Exchange Backend
3. Зберегти в безпечному місці

Крок 5: Перевірити статус Tron daemon'а

В UI SHKeeper → **Status** має показувати:

- Tron daemon: **Connected**
- Block scanner: **Syncing** або **Synced**

```
# Або через API:  
curl -s http://localhost:5100/api/v1/USDT/status \  
-H "X-Shkeeper-API-Key: <YOUR_API_KEY>" | python3 -m json.tool
```

Крок 6: Оновити Exchange Backend .env

Додати/оновити ці змінні в `.env` файлі Exchange Backend:

```
SHKEEPER_BASE_URL=http://localhost:5100          # або http://shkeeper:5000 при  
спільній мережі  
SHKEEPER_API_KEY=<ключ з кроку 4>  
SHKEEPER_PAYOUT_LOGIN=admin  
SHKEEPER_PAYOUT_PASSWORD=<пароль з кроку 2>  
SHKEEPER_WEBHOOK_CALLBACK_URL=http://buysell-service:4337/webhooks/shkeeper/deposit  
SHKEEPER_PAYOUT_CALLBACK_URL=http://buysell-service:4337/webhooks/shkeeper/payout
```

12. Перемикання мереж (Testnet ↔ Mainnet)

Tron Nile Testnet → Mainnet

⚠ **УВАГА:** Перемикання мережі видаляє всі гаманці та дані Tron daemon'a!

```
cd /opt/shkeeper

# 1. Зупинити контейнери
docker compose down

# 2. Видалити дані Tron (гаманці прив'язані до мережі)
docker volume rm shkeeper_shkeeper-tron-data
# Можна також видалити shkeeper-data якщо потрібен повний ресет:
# docker volume rm shkeeper_shkeeper-data

# 3. Змінити environment в docker-compose.yml для OBOX сервісів
# (tron-shkeeper та tron-shkeeper-celery):
#   - TRON_NETWORK=main          # Було: nile
#   - FULLNODE_URL=http://fullnode.tron.shkeeper.io # Було:
https://nile.trongrid.io

# 4. Перебїлдити та запустити
docker compose up -d --build

# 5. Зайти в UI → встановити пароль → налаштувати wallet encryption

# 6. Отримати новий API Key та оновити SHKEEPER_API_KEY в Exchange Backend .env
```

Конфігурації мереж:

Параметр	Mainnet	Testnet (Nile)
TRON_NETWORK	main	nile
FULLNODE_URL	http://fullnode.tron.shkeeper.io	https://nile.trongrid.io
USDT Contract	TR7NHqjeKQxGTCi8q8ZY4pL8otSzgJLj6t (авто)	TXYZopYRdj2D9XRtbG411XZZ3kM5VkAeBf (авто)

Contract addresses для USDT/USDC зашиті в коді tron-shkeeper (`app/config.py`) і автоматично підбираються по `TRON_NETWORK` .

13. Моніторинг та логування

Перевірка стану контейнерів

```
# Статус всіх контейнерів
docker compose ps
```

```
# Використання ресурсів
docker stats --no-stream --format "table {{.Name}}\t{{.MemUsage}}\t{{.CPUPerc}}" \
  shkeeper shkeeper-redis tron-shkeeper tron-shkeeper-celery
```

Логи

```
# Головний SHKeeper (API запити, webhooks)
docker logs shkeeper -f --tail=100

# Tron даємон (сканування блоків, транзакції)
docker logs tron-shkeeper -f --tail=100

# Celery worker (payouts, фонові задачі)
docker logs tron-shkeeper-celery -f --tail=100

# Redis
docker logs shkeeper-redis -f --tail=50

# Всі контейнери одразу
docker compose logs -f --tail=50
```

Health-check endpoints

Endpoint	Метод	Що перевіряє
http://localhost:5100/	GET	SHKeeper UI доступний
http://localhost:5100/api/v1/USDT/status	GET (+API key header)	Стан синхронізації Tron блокчейну
http://localhost:5100/api/v1/USDT/balance	GET (+API key header)	Баланс USDT гаманця
http://localhost:5100/api/v1/TRX/balance	GET (+API key header)	Баланс TRX (gas)

Приклад health-check скрипта

```
#!/bin/bash
# /opt/shkeeper/healthcheck.sh

API_KEY="your-shkeeper-api-key"
BASE_URL="http://localhost:5100"

# Check SHKeeper UI
HTTP_CODE=$(curl -s -o /dev/null -w "%{http_code}" "$BASE_URL/")
if [ "$HTTP_CODE" != "200" ] && [ "$HTTP_CODE" != "302" ]; then
  echo "CRITICAL: SHKeeper UI is down (HTTP $HTTP_CODE)"
  exit 2
fi
```

```
# Check Tron sync status
STATUS=$(curl -s -H "X-Shkeeper-API-Key: $API_KEY" "$BASE_URL/api/v1/USDT/status")
SYNCED=$(echo "$STATUS" | python3 -c "import sys,json;
print(json.load(sys.stdin).get('synced', False))" 2>/dev/null)
if [ "$SYNCED" != "True" ]; then
    echo "WARNING: Tron is not synced yet"
    exit 1
fi

echo "OK: SHKeeper is healthy"
exit 0
```

14. Бекапи та відновлення

Що бекапити

Що	Docker volume	Вміст	Критичність
SHKeeper data	shkeeper-data	SQLite БД, конфіг, wallet metadata	 Високо
Tron data	shkeeper-tron-data	SQLite БД, приватні ключі гаманців, стан сканера	 Критично
Redis data	shkeeper-redis-data	Celery queue state	 Низько (відновлюється автоматично)

Бекап

```
# Зупинити контейнери перед бекапом (для консистентності)
docker compose stop

# Бекап volumes
BACKUP_DIR="/opt/backups/shkeeper/$(date +%Y%m%d_%H%M%S)"
mkdir -p "$BACKUP_DIR"

# Бекап shkeeper-data
docker run --rm -v shkeeper_shkeeper-data:/data -v "$BACKUP_DIR":/backup \
    alpine tar czf /backup/shkeeper-data.tar.gz -C /data .

# Бекап tron-data (КРИТИЧНО – містить приватні ключі!)
docker run --rm -v shkeeper_shkeeper-tron-data:/data -v "$BACKUP_DIR":/backup \
    alpine tar czf /backup/shkeeper-tron-data.tar.gz -C /data .

# Запустити контейнери
docker compose start

echo "Backup saved to: $BACKUP_DIR"
```


Відновлення

```
# Зупинити контейнери
docker compose down

# Видалити існуючі volumes
docker volume rm shkeeper_shkeeper-data shkeeper_shkeeper-tron-data

# Створити нові volumes
docker volume create shkeeper_shkeeper-data
docker volume create shkeeper_shkeeper-tron-data

# Відновити з бекапу
BACKUP_DIR="/opt/backups/shkeeper/20260324_120000" # ваша дата

docker run --rm -v shkeeper_shkeeper-data:/data -v "$BACKUP_DIR":/backup \
  alpine sh -c "cd /data && tar xzf /backup/shkeeper-data.tar.gz"

docker run --rm -v shkeeper_shkeeper-tron-data:/data -v "$BACKUP_DIR":/backup \
  alpine sh -c "cd /data && tar xzf /backup/shkeeper-tron-data.tar.gz"

# Запустити
docker compose up -d
```

⚠ **Tron data volume містить приватні ключі гаманців!** Бекапи мають зберігатися в зашифрованому вигляді, доступ обмежений.

15. Оновлення

Оновлення SHKeeper та Tron daemon

```
cd /opt/shkeeper

# 1. Стягнути останні зміни з upstream
git pull origin main

# 2. Оновити tron daemon
cd daemons/tron-shkeeper
git pull origin main
cd ../../

# 3. Перебудувати та перезапустити
docker compose up -d --build

# 4. Перевірити здоров'я
docker compose ps
docker logs shkeeper --tail=20
docker logs tron-shkeeper --tail=20
```

⚠ **Перед оновленням:** завжди робіть бекап volumes (особливо `shkeeper-tron-data`).

16. Troubleshooting

Проблема: Tron daemon не підключається до SHKeeper

Симптоми: В UI показує "Tron: Disconnected"

Рішення:

```
# Перевірити логи даемон'а
docker logs tron-shkeeper --tail=50

# Перевірити мережевий зв'язок
docker exec tron-shkeeper curl -s http://shkeeper:5000/ || echo "Cannot reach shkeeper"

# Перевірити що SHKEEPER_HOST правильний
docker exec tron-shkeeper env | grep SHKEEPER
```

Проблема: Wallet encryption pending

Симптоми: В логах: "Waiting for encryption password from shkeeper"

Рішення:

1. Зайти в UI SHKeeper → `/set-password` (якщо перший запуск)
2. Або зайти в Settings → Wallet Encryption → Enable/Skip

Проблема: Block scanner не синхронізується

Симптоми: Статус показує synced=false довше 30 хвилин

Рішення:

```
# Перевірити логи сканера
docker logs tron-shkeeper --tail=100 | grep -i "scanner\|block\|error"

# Перевірити доступність Tron ноди
docker exec tron-shkeeper curl -s "$FULLNODE_URL/wallet/getnowblock" | head -c 200

# Якщо нода недоступна — змінити FULLNODE_URL
```

Проблема: Celery worker не стартує

Симптоми: Контейнер `tron-shkeeper-celery` рестартує

Рішення:

```
# Перевірити логи
docker logs tron-shkeeper-celery --tail=50
```

```
# Найчастіша причина: відсутній C_FORCE_ROOT=1
# Перевірити:
docker exec tron-shkeeper-celery env | grep C_FORCE_ROOT
```

Проблема: Redis connection refused

Симптоми: Помилки "Connection refused" в логах daemon/celery

Рішення:

```
# Перевірити здоров'я Redis
docker compose ps shkeeper-redis
docker exec shkeeper-redis redis-cli ping # Має повернути PONG

# Перевірити hostname
docker exec tron-shkeeper getent hosts shkeeper-redis
```

Проблема: Порт 5100 зайнятий

```
# Знайти що займає порт
ss -tlnp | grep :5100

# Або змінити порт в docker-compose.yml:
# ports:
#   - '5101:5000'    # ← замінити 5100 на інший
```

Проблема: TRON_NETWORK invalid value

Симптоми: ValueError: 'mainnet' is not a valid TronNetwork

Рішення: Допустимі значення **тільки** main або nile (не mainnet, не testnet).

17. Чеклісти

✅ Чекліст перед першим запуском

- ☐ Docker і Docker Compose встановлені
- ☐ Порти 5100 та 6380 вільні
- ☐ Код сконовано (shkeeper.io + tron-shkeeper)
- ☐ docker-compose.yml створено та перевірено
- ☐ TRON_NETWORK та FULLNODE_URL відповідають потрібній мережі
- ☐ DEV_MODE=0 для production (або 1 для testing)

✅ Чекліст після запуску

- ☐ Всі 4 контейнери у стані Up
- ☐ SHKeeper UI доступний на http://<server>:5100
- ☐ Пароль адміністратора встановлено
- ☐ Wallet encryption налаштовано

- ☐ API Key отримано та записано
- ☐ Tron daemon показує "Connected" в UI
- ☐ Block scanner синхронізується

✅ Чекліст інтеграції з Exchange Backend

- ☐ `SHKEEPER_BASE_URL` прописано в `.env` Exchange Backend
- ☐ `SHKEEPER_API_KEY` відповідає ключу в UI SHKeeper
- ☐ `SHKEEPER_PAYOUT_LOGIN` / `SHKEEPER_PAYOUT_PASSWORD` відповідають паролю адміна
- ☐ `SHKEEPER_WEBHOOK_CALLBACK_URL` доступний з контейнера SHKeeper
- ☐ `SHKEEPER_PAYOUT_CALLBACK_URL` доступний з контейнера SHKeeper
- ☐ GitHub Secrets оновлені для dev/release середовищ
- ☐ Тест: створити payment request через API → перевірити що адреса згенерована

✅ Чекліст безпеки (production)

- ☐ `DEV_MODE=0`
- ☐ SHKeeper UI закритий за nginx з IP-whitelist або VPN
- ☐ Бекапи volumes налаштовані (особливо `shkeeper-tron-data` — приватні ключі!)
- ☐ Пароль адміністратора надійний (не `devpassword123`)
- ☐ `FULLNODE_URL` вказує на надійну ноду (не публічну з rate-limits)
- ☐ Redis не виставлений назовні (порт 6380 тільки для debug, в prod можна прибрати)